Week_3_Milestone 2: White-Paper Draft
Velazquez-Feliciano, Adrian
DSC 680: Applied Data Science
04/05/2026

Milestone 2: White-Paper Draft

Title: Customer Segmentation to Improve Marketing Efficiency and Campaign Targeting

Business Problem

Many businesses use broad marketing strategies that treat all customers the same, even though customers differ in income, spending habits, purchasing frequency, and responsiveness to campaigns. This limits the effectiveness of promotions and may cause companies to waste marketing resources on customers who are unlikely to respond. The business problem addressed in this project is how to identify meaningful customer groups using historical customer data so that a company can better target marketing efforts, improve retention, and increase campaign efficiency. By segmenting customers into groups with similar purchasing and behavioral characteristics, businesses can create more personalized and cost effective strategies.

Background and History

Customer segmentation is a widely used technique in business analytics and marketing. The goal is to divide a customer base into smaller groups that share similar characteristics, allowing businesses to better understand who their customers are and how they behave. Rather than marketing to everyone in the same way, firms can use segmentation to tailor promotions, communication, and product recommendations. In data science, clustering is one of the most common methods used for segmentation because it identifies patterns in data without needing predefined labels. K means clustering is especially popular because it is relatively simple, efficient, and effective for grouping customers with similar numeric characteristics. For this project, customer segmentation was used to analyze income, spending, purchases, and engagement metrics in order to identify actionable customer groups.

Data Explanation

The dataset used in this project was a customer segmentation dataset containing demographic, household, and purchasing information. The original dataset included variables such as birth year, education, marital status, income, household composition, recency of purchases, spending by product category, purchase behavior across channels,

and campaign response history. Several new variables were created during data preparation, including Total Spending, Total Purchases, Age, Total Children, and Customer Tenure Days. These derived variables made it easier to summarize customer behavior and compare segments. Records with unrealistic ages were removed, and missing income values were handled during preprocessing. The final dataset was suitable for clustering because it contained multiple behavioral and financial indicators that could be standardized and compared across customers.

Methods

This project used descriptive analysis, feature engineering, data standardization, K means clustering, and visualization. First, the dataset was explored using summary statistics and charts to understand customer spending and purchasing patterns. Next, derived features were created to better reflect overall customer value and engagement. The clustering analysis focused on variables such as income, recency, total spending, total purchases, web visits, age, and number of children in the household. These variables were scaled using StandardScaler so that no single measure would dominate the clustering process. The elbow method and silhouette score were used to evaluate cluster quality and support the selection of four clusters. Principal component analysis was then used to visualize the clusters in two dimensions.

Analysis

The analysis identified four distinct customer segments. Cluster 1 represented the highest value customers, with an average income of 78165.75, average spending of 1341.26, and the highest campaign response rate at 0.24. These customers also had the fewest monthly web visits, suggesting they may be efficient buyers who already know what they want. Cluster 3 represented another strong segment, with average income of 61218.34 and average spending of 895.84. This cluster also showed high purchase activity and moderate response rates. Cluster 0 and Cluster 2 were lower value groups. Cluster 0 had average income of 36938.64, very high recency at 74.98, low spending of 133.22, and very low response at 0.04. Cluster 2 had the lowest income at 34818.09 and similarly low spending at 121.08, but showed somewhat better response behavior at 0.16. Category level spending profiles showed that Cluster 1 spent the most across all product categories, especially wines and meat products, while Cluster 3 ranked second. These results suggest that the business has at least two high opportunity segments and two lower value segments requiring different engagement strategies.

Conclusion

The findings show that customer segmentation can provide meaningful business insight by identifying groups with different levels of value, engagement, and marketing responsiveness. The most important result is that not all customers should be approached in the same way. High value customers can be retained through premium offers and

loyalty strategies, while lower value segments may require re engagement or more cost conscious promotions. Overall, the analysis demonstrates that clustering can support smarter marketing decisions and improve how companies allocate resources.

Assumptions

This analysis assumed that the dataset is a reasonable representation of customer behavior and that the available variables capture important differences in purchasing patterns. It also assumed that K means clustering is appropriate for this type of data and that four clusters provide a useful balance between statistical separation and business interpretability. Another assumption is that historical customer behavior can help inform future marketing strategy.

Limitations

This project has several limitations. The dataset does not include all possible factors influencing customer behavior, such as geographic variation, seasonality, product preferences over time, or competitor actions. The data appears to represent a snapshot rather than a full longitudinal customer history, which limits the ability to analyze change over time. In addition, K means clustering requires the number of clusters to be chosen in advance and can be sensitive to scaling and outliers.

Challenges

One challenge in this project was selecting the most relevant variables for clustering. Including too many variables can reduce interpretability, while including too few may oversimplify customer differences. Another challenge was ensuring that the final clusters were not only statistically separate but also useful from a business perspective. Balancing mathematical output with real world interpretability was an important part of the project.

Future Uses and Additional Applications

This project could be extended by using additional customer data such as geographic region, online browsing history, promotion exposure, or product level transaction details. Future analyses could compare segmentation results over time or test whether the same clusters remain stable across multiple campaigns. The model could also be expanded using other machine learning methods such as hierarchical clustering or predictive modeling for customer response and retention.

Recommendations

Based on the results, the business should prioritize Cluster 1 and Cluster 3 for personalized marketing and retention strategies because these groups generate the highest spending and strongest purchase activity. Cluster 0 may benefit from re engagement campaigns because of its high recency and low response rate. Cluster 2 may respond better to value based promotions because it has lower income and low overall spending but a somewhat better response rate than Cluster 0. Marketing resources should be

allocated differently across these groups instead of using a one size fits all campaign approach.

Implementation Plan

A practical implementation plan would begin with assigning each active customer to a segment based on the clustering model. The marketing team could then design different campaigns for each segment, such as loyalty rewards for high value customers and discount driven outreach for lower value groups. Dashboards should be created to monitor spending, response rates, and retention by segment. The segmentation model should be reviewed and refreshed on a regular basis so that it continues to reflect current customer behavior. Finally, campaign performance should be measured using A B testing to confirm that segmentation based strategies improve outcomes.

Ethical Considerations

Customer segmentation raises important ethical concerns. Even when direct identifiers are removed, customer data still reflects personal behavior and should be handled responsibly. Privacy protection is essential, and customer information should only be used for legitimate business purposes. Another concern is bias in interpretation. If segments are described in ways that unfairly stereotype people based on income, age, family structure, or purchasing power, the analysis could reinforce harmful assumptions. Businesses must be careful not to use segmentation to unfairly exclude groups from opportunities or manipulate vulnerable customers. In this project, segmentation should be viewed as a tool for improving customer experience and marketing efficiency, not for discrimination.

Appendix A - Cluster Summary

Cluster 0 is a low income, low spending, and low response segment with high recency. Cluster 1 is the highest value segment, with the highest income, total spending, and campaign response. Cluster 2 is another lower value segment with lower income and spending but slightly better response than Cluster 0. Cluster 3 is a strong middle to upper tier segment with high spending and high purchase activity. Category level results show that Cluster 1 leads spending across all product categories and Cluster 3 ranks second.

References

Dapat, V. (n.d.). Customer segmentation clustering. Kaggle.

Data Science with Onur. (2025, July 19). Customer segmentation using machine learning K means clustering full Python data science project [Video]. YouTube.

Appendix A. Supporting Documentation

This appendix provides supporting documentation for the customer segmentation analysis conducted in this project. The purpose of the appendix is to summarize the data preparation steps, derived variables, clustering inputs, and final outputs used to support the findings presented in the white paper.

Dataset Overview

The dataset used for this project contains customer demographic, purchasing, and campaign response information. Variables include income, education, marital status, number of children in the household, enrollment date, recency, product spending, purchase channels, website visits, and campaign response indicators.

Data Preparation

Several preprocessing steps were performed before clustering. Missing values were removed from the dataset to ensure a clean analytical sample. The **Dt_Customer** field was converted to datetime format for time-based calculations. Unreasonable age values were filtered out by calculating age from year of birth and restricting the sample to adult customers within a realistic range.

Feature Engineering

New variables were created to improve the analysis. **Total_Spending** was calculated as the sum of spending across product categories, including wine, fruits, meat products, fish products, sweet products, and gold products. **Total_Purchases** was calculated as the combined number of web, catalog, and store purchases. **Age** was derived from year of birth. **Customer_Since** was created from the customer enrollment date, and **Total_Children** was calculated as the sum of **Kidhome** and **Teenhome**.

Variables Used for Clustering

The clustering analysis was performed using the following variables:

- Income
- Recency
- Total_Spending
- NumWebPurchases
- NumCatalogPurchases
- NumStorePurchases

- NumWebVisitsMonth

These variables were selected because they reflect both customer value and customer engagement across purchasing channels.

Standardization and Model Selection

Because the clustering variables were measured on different scales, the data was standardized using **StandardScaler** before fitting the model. The elbow method and silhouette score were used to compare different cluster solutions. Based on these diagnostics and the interpretability of the results, a four-cluster solution was selected.

Final Cluster Outputs

The final analysis produced a cluster assignment for each customer and generated summary tables showing the average characteristics of each cluster. These outputs included:

- **customer_segmentation_clustered_output.csv**
- **cluster_profile_summary.csv**
- **cluster_category_profile.csv**

Interpretation Support

The cluster summaries showed meaningful differences in income, spending, purchases, web activity, family structure, and response behavior. In particular, one segment had the highest average income and spending, while another high-performing segment showed strong purchasing activity with somewhat different browsing behavior. Lower-spending clusters showed lower purchase volume and weaker overall customer value.

Visual Support Included in the White Paper

The white paper includes several illustrations created from the analysis, such as:

- Income distribution
- Total spending distribution
- Income versus total spending scatterplot
- Elbow method plot
- Average total spending by cluster
- Average campaign response rate by cluster
- PCA cluster visualization

These visuals were used to support interpretation of the customer segments and demonstrate how the final clustering solution was chosen.

10 Questions an Audience Would Ask You

1. **Why did you choose customer segmentation for this project?**
2. **What business problem does this project solve?**
3. **Why did you use K-means clustering?**
4. **How did you decide on four clusters?**
5. **What variables were most important in creating the clusters?**
6. **What did the clusters reveal about the customers?**
7. **Which customer segment is the most valuable?**
8. **How can a business use these clustering results?**
9. **What are the limitations of this analysis?**
10. **What would you improve if you continued this project?**

Project 1

Velazquez-Feliciano, Adrian

DSC 680: Applied Data Science

04/05/2026

In [1]:
```
# Title: Customer Segmentation using KMeans
# Author: Vishakh Patel
# Date: 05 April 2026
# Modified By: Adrian Velazquez-Feliciano
# Description: This program loads, cleans, and analyzes customer data to perform
# customer segmentation using K-means clustering. It creates summary statistics,
# visualizations, and cluster profiles to support marketing recommendations.
#
# Code Attribution:
# This project was developed by Adrian Velazquez-Feliciano for DSC 680.
# The overall analysis, data preparation, feature engineering, clustering,
# interpretation, and written deliverables were completed for this assignment.
#
# Some ideas for workflow, clustering structure, and visualization were inspired by:
# - Course materials and examples from DSC 680
# - The Kaggle Customer Segmentation dataset page
# - General Python documentation for pandas, matplotlib, seaborn, and scikit-learn
# - Public educational video tutorials on customer segmentation and K-means clustering
#
# Specific outside inspiration referenced in the paper:
# - Dapat, V.: Customer segmentation clustering. Kaggle.
# - Vishakh, Patel: Customer Segmentation using KMeans. Kaggle
# - Data Science with Onur. Customer segmentation using machine learning
#   K-means clustering full Python data science project. YouTube. 19 July 2025.
#
```

```
# Any code adapted from examples was rewritten, modified, and integrated
# for the specific goals of this project.
# ==========================================
```

Import Libraries, read data, and preview dataframe

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn as skl
from tqdm import tqdm
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

In [3]:
```
# Confirm my working directory
import os
print(os.getcwd())
```

```
C:\Users\velaz\Desktop\JupyterProject
```

In [4]:
```
# Read the customer segmentation dataset into a pandas DataFrame
file_path = "customer_segmentation.csv"
df = pd.read_csv(file_path)

# Display the first five rows so the structure of the dataset can be reviewed
print(df.head())
```

```
     ID  Year_Birth   Education Marital_Status   Income  Kidhome  Teenhome  \
0  5524        1957  Graduation         Single  58138.0        0         0
1  2174        1954  Graduation         Single  46344.0        1         1
2  4141        1965  Graduation       Together  71613.0        0         0
3  6182        1984  Graduation       Together  26646.0        1         0
4  5324        1981         PhD        Married  58293.0        1         0

  Dt_Customer  Recency  MntWines  ...  NumWebVisitsMonth  AcceptedCmp3  \
0  04-09-2012       58       635  ...                  7             0
1  08-03-2014       38        11  ...                  5             0
2  21-08-2013       26       426  ...                  4             0
3  10-02-2014       26        11  ...                  6             0
4  19-01-2014       94       173  ...                  5             0

   AcceptedCmp4  AcceptedCmp5  AcceptedCmp1  AcceptedCmp2  Complain  \
0             0             0             0             0         0
1             0             0             0             0         0
2             0             0             0             0         0
3             0             0             0             0         0
4             0             0             0             0         0

   Z_CostContact  Z_Revenue  Response
0              3         11         1
1              3         11         0
2              3         11         0
3              3         11         0
4              3         11         0

[5 rows x 29 columns]
```

Data Cleaning and Feature Engineering

In [5]:
```
# Display dataset information to review column names, data types, and missing values
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   2240 non-null   int64
 1   Year_Birth           2240 non-null   int64
 2   Education            2240 non-null   str
 3   Marital_Status       2240 non-null   str
 4   Income               2216 non-null   float64
 5   Kidhome              2240 non-null   int64
 6   Teenhome             2240 non-null   int64
 7   Dt_Customer          2240 non-null   str
 8   Recency              2240 non-null   int64
 9   MntWines             2240 non-null   int64
 10  MntFruits            2240 non-null   int64
 11  MntMeatProducts      2240 non-null   int64
 12  MntFishProducts      2240 non-null   int64
 13  MntSweetProducts     2240 non-null   int64
 14  MntGoldProds         2240 non-null   int64
 15  NumDealsPurchases    2240 non-null   int64
 16  NumWebPurchases      2240 non-null   int64
 17  NumCatalogPurchases  2240 non-null   int64
 18  NumStorePurchases    2240 non-null   int64
 19  NumWebVisitsMonth    2240 non-null   int64
 20  AcceptedCmp3         2240 non-null   int64
 21  AcceptedCmp4         2240 non-null   int64
 22  AcceptedCmp5         2240 non-null   int64
 23  AcceptedCmp1         2240 non-null   int64
 24  AcceptedCmp2         2240 non-null   int64
 25  Complain             2240 non-null   int64
 26  Z_CostContact        2240 non-null   int64
 27  Z_Revenue            2240 non-null   int64
 28  Response             2240 non-null   int64
dtypes: float64(1), int64(25), str(3)
memory usage: 507.6 KB
```

In [6]:
```
# Drop missing values
df = df.dropna()
```

In [7]:
```
# Investigate the values with string format for possible formatting
```

In [8]:
```
# Marital_Status values analysis
df['Marital_Status'].value_counts()
```

Out[8]:
```
Marital_Status
Married     857
Together    573
Single      471
Divorced    232
Widow        76
Alone         3
Absurd        2
YOLO          2
Name: count, dtype: int64
```

In [9]:
```
# Education values analysis
df['Education'].value_counts()
```

Out[9]:
```
Education
Graduation    1116
PhD            481
Master         365
2n Cycle       200
Basic           54
Name: count, dtype: int64
```

In [10]:
```
# DT_Customer values analysis
df['Dt_Customer'].value_counts()
```

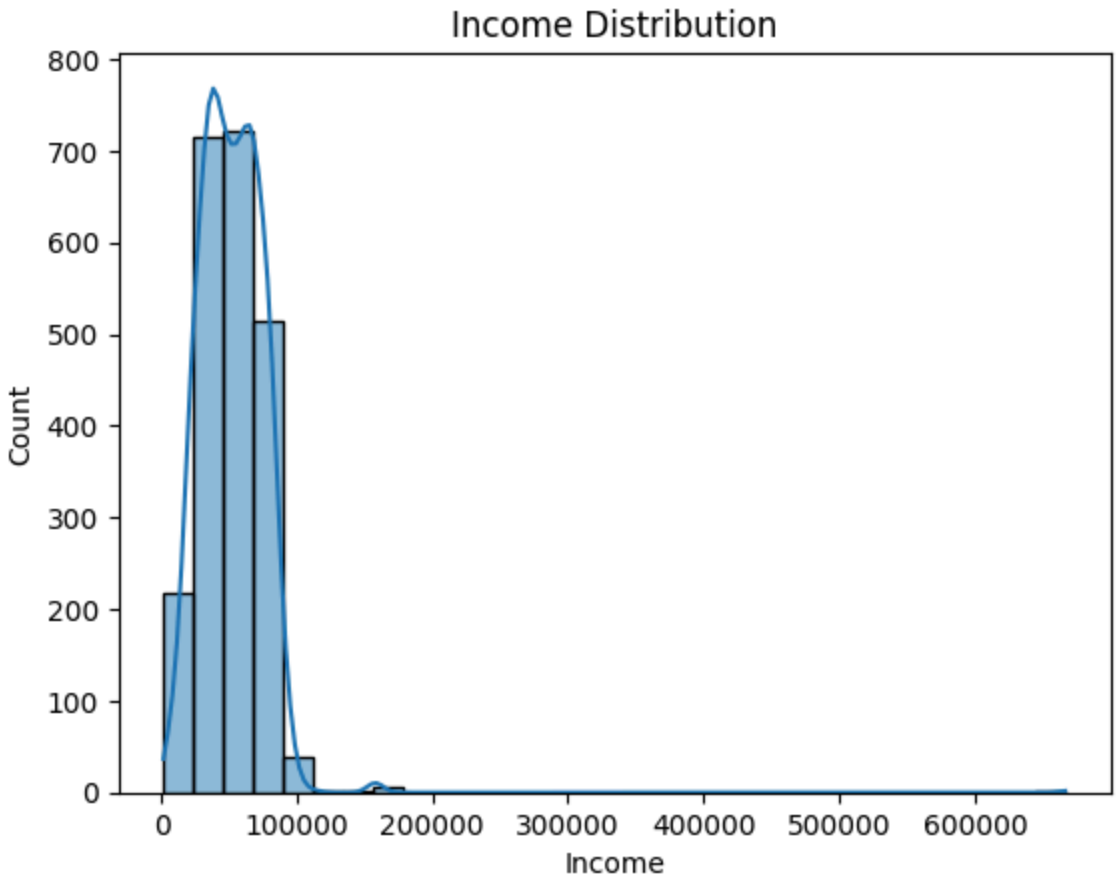
Out[10]:
```
Dt_Customer
31-08-2012    12
12-05-2014    11
12-09-2012    11
14-02-2013    11
22-05-2014    10
              ..
01-11-2013     1
10-08-2013     1
11-10-2012     1
20-12-2012     1
09-01-2014     1
Name: count, Length: 662, dtype: int64
```

In [11]:
```
# Convert the customer enrollment date column to datetime format
# This is necessary for time-based calculations later in the analysis
df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'], dayfirst = True)
# Confirm changes
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 2216 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   2216 non-null   int64
 1   Year_Birth           2216 non-null   int64
 2   Education            2216 non-null   str
 3   Marital_Status       2216 non-null   str
 4   Income               2216 non-null   float64
 5   Kidhome              2216 non-null   int64
 6   Teenhome             2216 non-null   int64
 7   Dt_Customer          2216 non-null   datetime64[us]
 8   Recency              2216 non-null   int64
 9   MntWines             2216 non-null   int64
 10  MntFruits            2216 non-null   int64
 11  MntMeatProducts      2216 non-null   int64
 12  MntFishProducts      2216 non-null   int64
 13  MntSweetProducts     2216 non-null   int64
 14  MntGoldProds         2216 non-null   int64
 15  NumDealsPurchases    2216 non-null   int64
 16  NumWebPurchases      2216 non-null   int64
 17  NumCatalogPurchases  2216 non-null   int64
 18  NumStorePurchases    2216 non-null   int64
 19  NumWebVisitsMonth    2216 non-null   int64
 20  AcceptedCmp3         2216 non-null   int64
 21  AcceptedCmp4         2216 non-null   int64
 22  AcceptedCmp5         2216 non-null   int64
 23  AcceptedCmp1         2216 non-null   int64
 24  AcceptedCmp2         2216 non-null   int64
 25  Complain             2216 non-null   int64
 26  Z_CostContact        2216 non-null   int64
 27  Z_Revenue            2216 non-null   int64
 28  Response             2216 non-null   int64
dtypes: datetime64[us](1), float64(1), int64(25), str(2)
memory usage: 519.4 KB
```

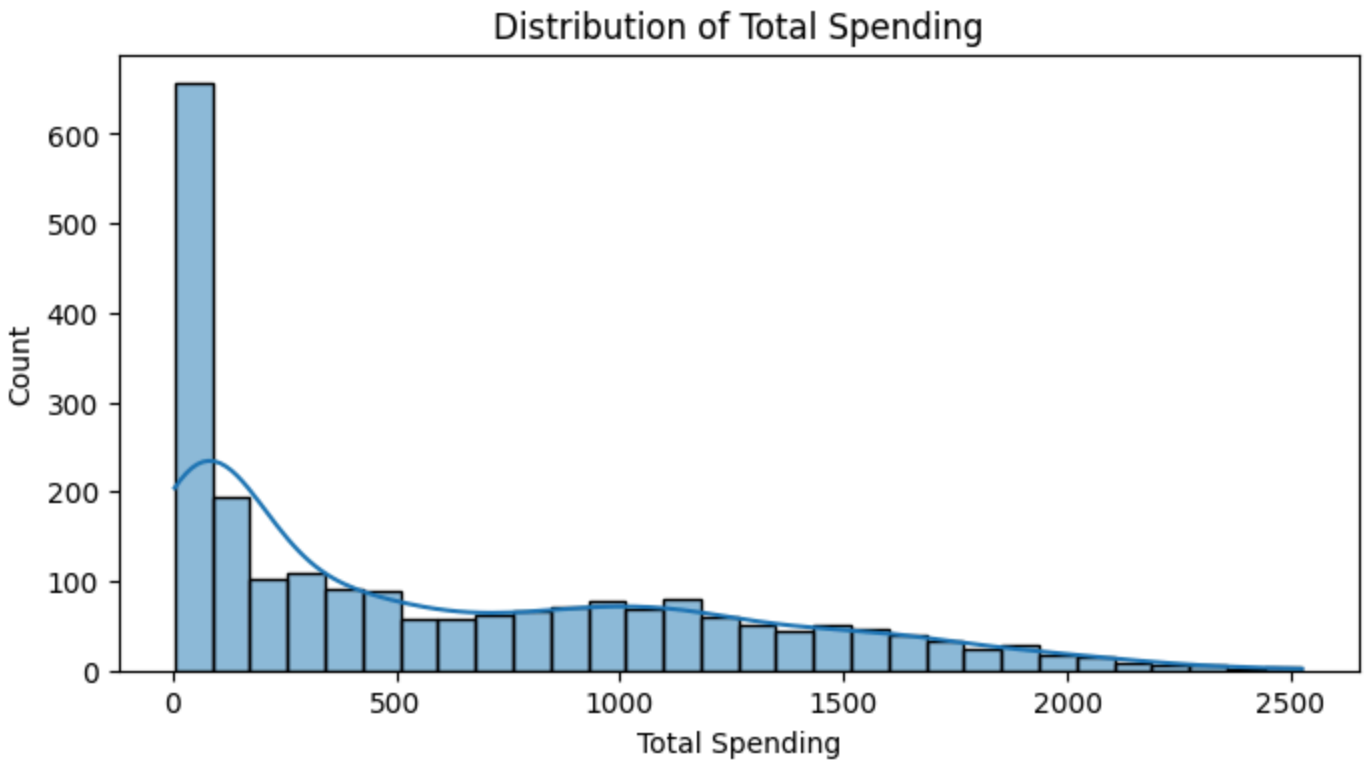
Make some quick overview visuals

In [12]:
```
## Income Distribution
sns.histplot(df['Income'], bins = 30, kde=True)
plt.title('Income Distribution')
plt.show()
```



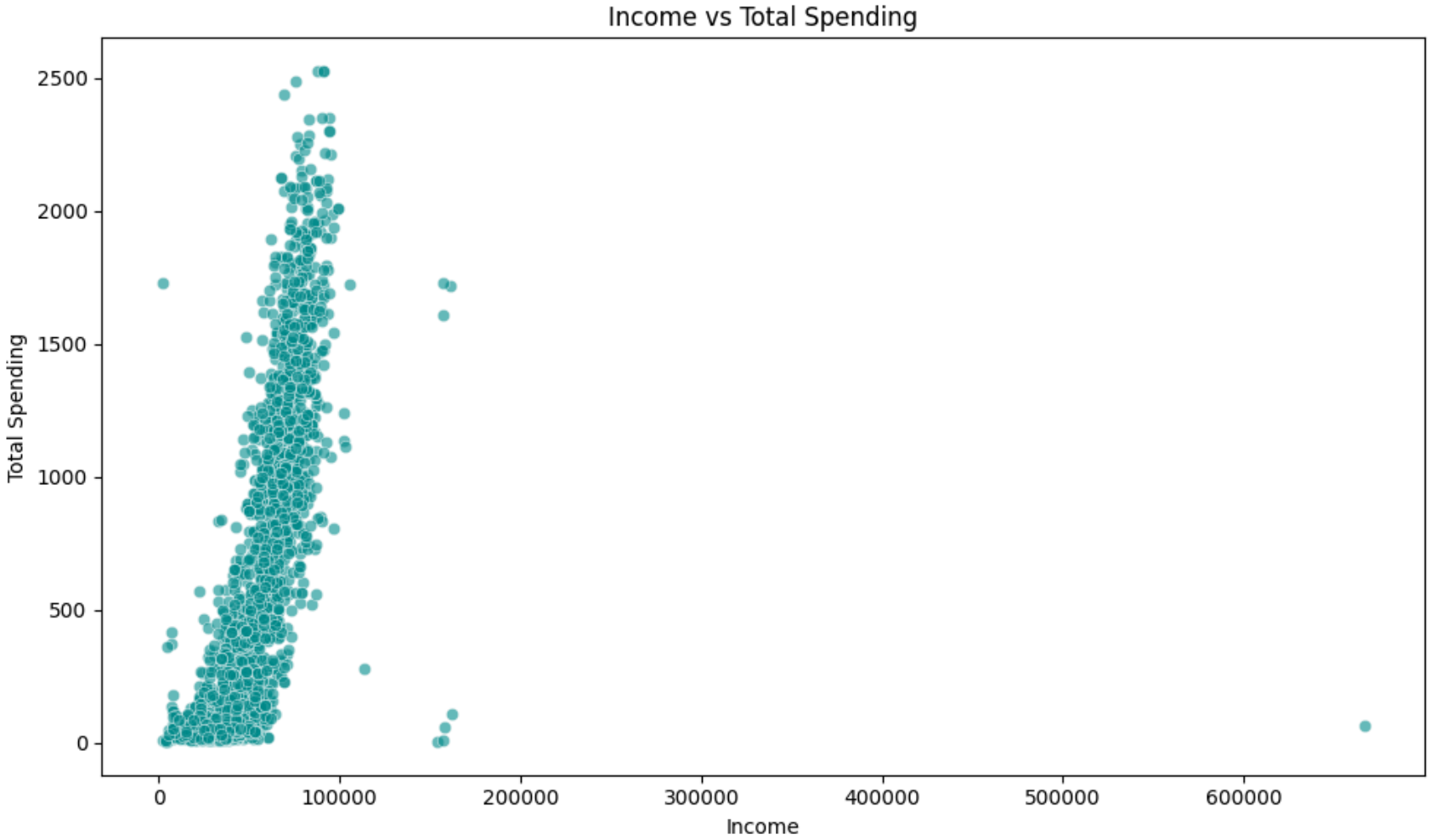
In [13]:
```
## Total Spending Distribution
spend_cols = ['MntWines','MntFruits','MntMeatProducts','MntFishProducts','MntSweetProducts','MntGoldProds']
purchase_cols = [
    "NumWebPurchases",
    "NumCatalogPurchases",
    "NumStorePurchases"
]
df['Total_Spending'] = df[spend_cols].sum(axis=1)
df["Total_Purchases"] = df[purchase_cols].sum(axis=1)
```

```
plt.figure(figsize=(8,4))
sns.histplot(df['Total_Spending'], bins=30, kde=True)
plt.title('Distribution of Total Spending')
plt.xlabel('Total Spending')
plt.ylabel('Count')
plt.show()
```



In [14]:
```
# Income vs Total Spending
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data= df,
    x="Income",
    y="Total_Spending",
    alpha=0.6,
    color="darkcyan"
)
plt.title("Income vs Total Spending")
plt.xlabel("Income")
```

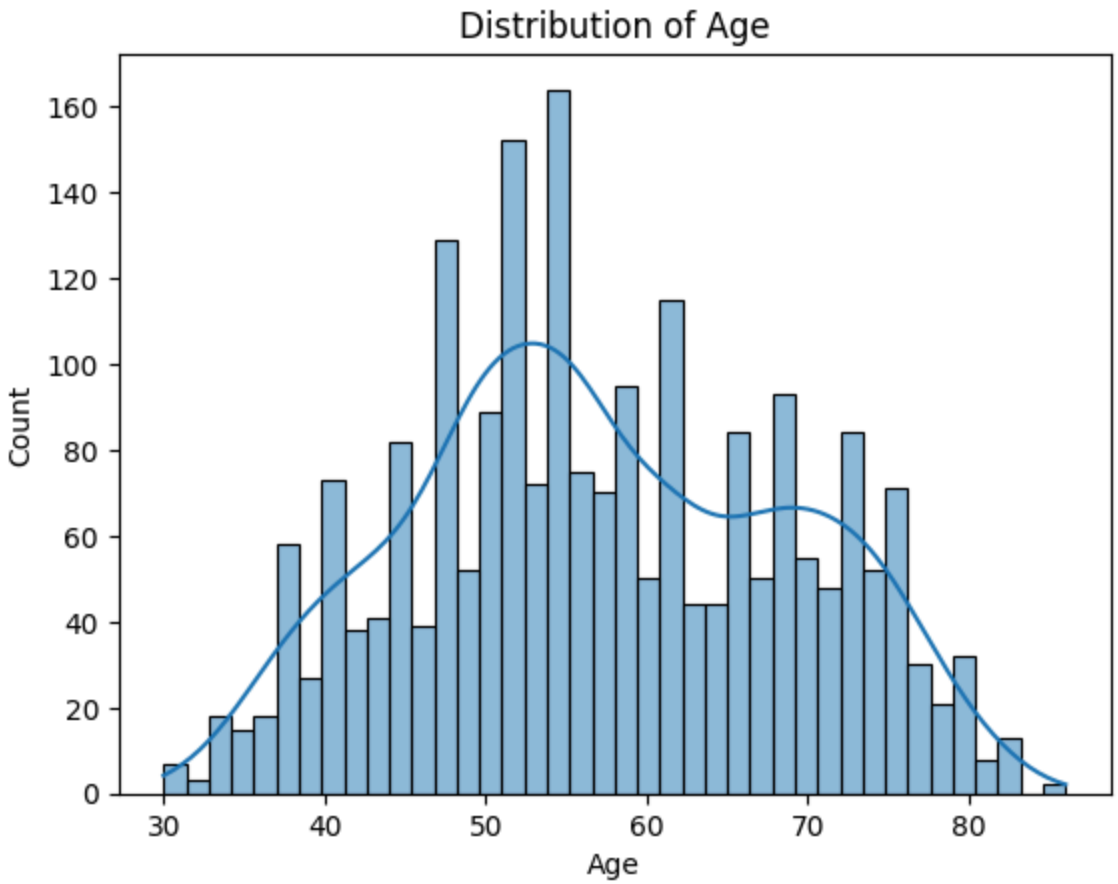
```
plt.ylabel("Total Spending")
plt.tight_layout()
plt.show()
```



In [15]:
```
# Create an Age feature
df["Age"] = 2026 - df["Year_Birth"]
# Remove unrealistic ages to improve data quality before analysis
df = df[(df["Age"] >= 18) & (df["Age"] <= 100)].copy()
# Plot the Age Distribution after removing unrealistic values
sns.histplot(df['Age'], bins=40, kde=True)
plt.title('Distribution of Age')
```

```
plt.xlabel('Age')
plt.ylabel('Count')
```

Out[15]: Text(0, 0.5, 'Count')



In [16]:
```
# Create a Total_Children feature by combining children and teenagers in the household
# This simplifies household composition into one variable
df["Total_Children"] = df['Kidhome'] + df['Teenhome']
```

In [17]:
```
# Group customers by assigned cluster and calculate average values
# This summary helps interpret the characteristics of each customer segment
summary_cols = [
    "Income",
    "Age",
```

```
    "Recency",
    "Total_Spending",
    "Total_Purchases",
    "NumWebVisitsMonth",
    "Total_Children",
    "Response"
]
# Generate descriptive statistics for the selected variables
summary_stats = df[summary_cols].describe().round(2)
# Display the summary statistics table
print(summary_stats)
```

```
         Income      Age  Recency  Total_Spending  Total_Purchases  \
count    2213.00  2213.00  2213.00         2213.00          2213.00
mean    52236.58    57.08    49.01          607.02            12.56
std     25178.60    11.70    28.94          602.49             7.20
min      1730.00    30.00     0.00            5.00             0.00
25%     35246.00    49.00    24.00           69.00             6.00
50%     51373.00    56.00    49.00          397.00            12.00
75%     68487.00    67.00    74.00         1048.00            18.00
max    666666.00    86.00    99.00         2525.00            32.00

       NumWebVisitsMonth  Total_Children  Response
count            2213.00         2213.00   2213.00
mean                5.32            0.95      0.15
std                 2.43            0.75      0.36
min                 0.00            0.00      0.00
25%                 3.00            0.00      0.00
50%                 6.00            1.00      0.00
75%                 7.00            1.00      0.00
max                20.00            3.00      1.00
```

Clustering

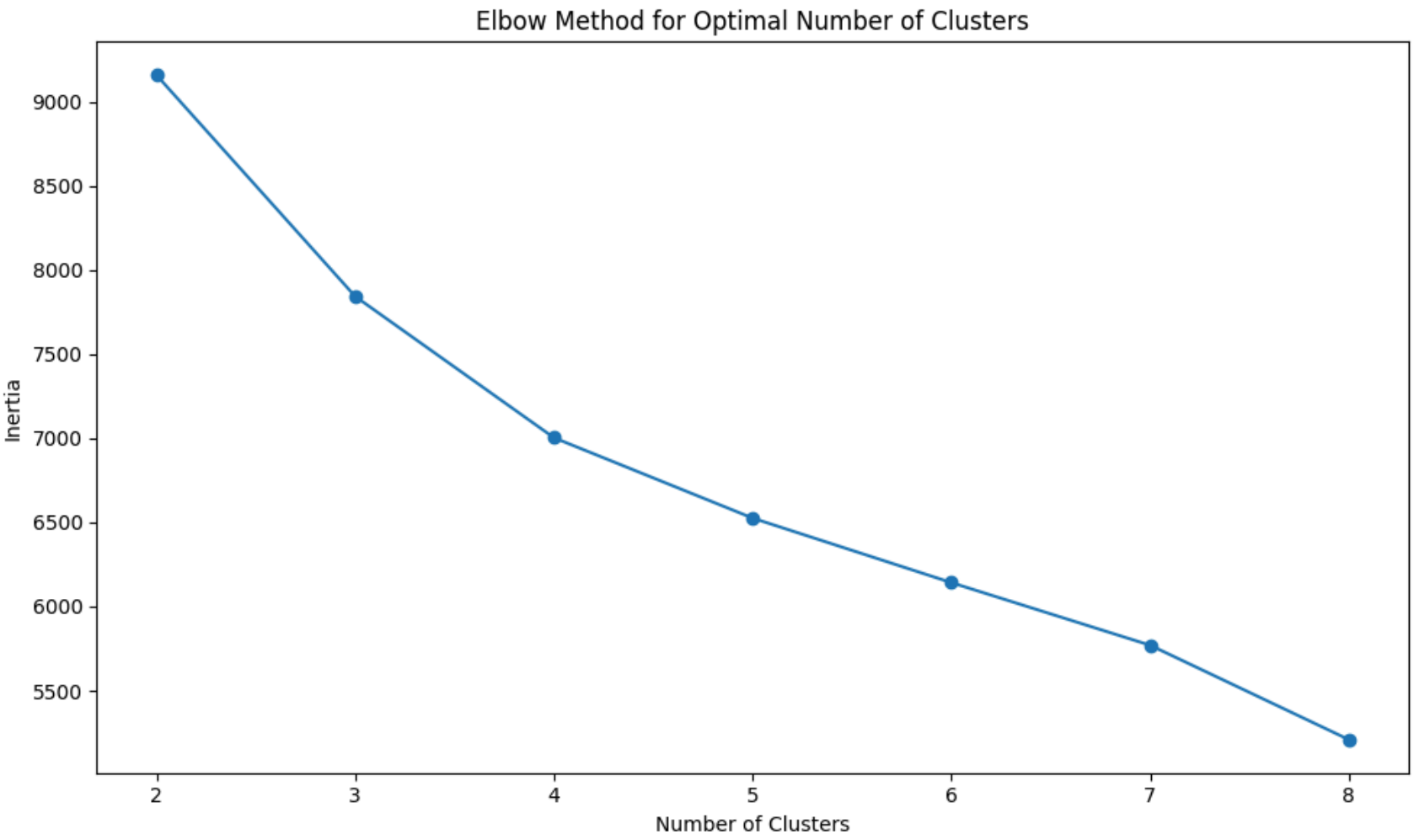
In [18]:

```
# Select the features used for clustering
cluster_features = [
    "Income",
    "Recency",
    "Total_Spending",
    "NumWebPurchases",
```

```
    "NumCatalogPurchases",
    "NumStorePurchases",
    "NumWebVisitsMonth"
]
# Create a separate DataFrame with only the clustering features
cluster_df = df[cluster_features].copy()

# Scale data

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_df)
```

In [19]:

```
# Use elbow method to compare inertia across cluster counts
inertia_vals = []
cluster_range = range(2, 9)

for k in cluster_range:
    kmeans_model = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_model.fit(cluster_scaled)
    inertia_vals.append(kmeans_model.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(cluster_range, inertia_vals, marker="o")
plt.title("Elbow Method for Optimal Number of Clusters")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.tight_layout()
plt.show()
```



In [20]:
```
# Compare silhouette scores for possible cluster counts
silhouette_vals = []
cluster_range = range(2, 9)

for k in cluster_range:
    kmeans_model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans_model.fit_predict(cluster_scaled)
    score = silhouette_score(cluster_scaled, labels)
    silhouette_vals.append(score)

silhouette_results = pd.DataFrame({
```

```
    "Clusters": list(cluster_range),
    "Silhouette_Score": silhouette_vals
})

print(silhouette_results)
```

```
   Clusters  Silhouette_Score
0         2          0.383526
1         3          0.307341
2         4          0.233066
3         5          0.222931
4         6          0.219361
5         7          0.219832
6         8          0.224236
```

In [21]:

```
# Fit final KMeans model
final_k = 4
kmeans_final = KMeans(n_clusters=final_k, random_state=42, n_init=10)
df["Cluster"] = kmeans_final.fit_predict(cluster_scaled)

# Check cluster counts
print(df["Cluster"].value_counts().sort_index())
```

```
Cluster
0    557
1    538
2    586
3    532
Name: count, dtype: int64
```

In [22]:

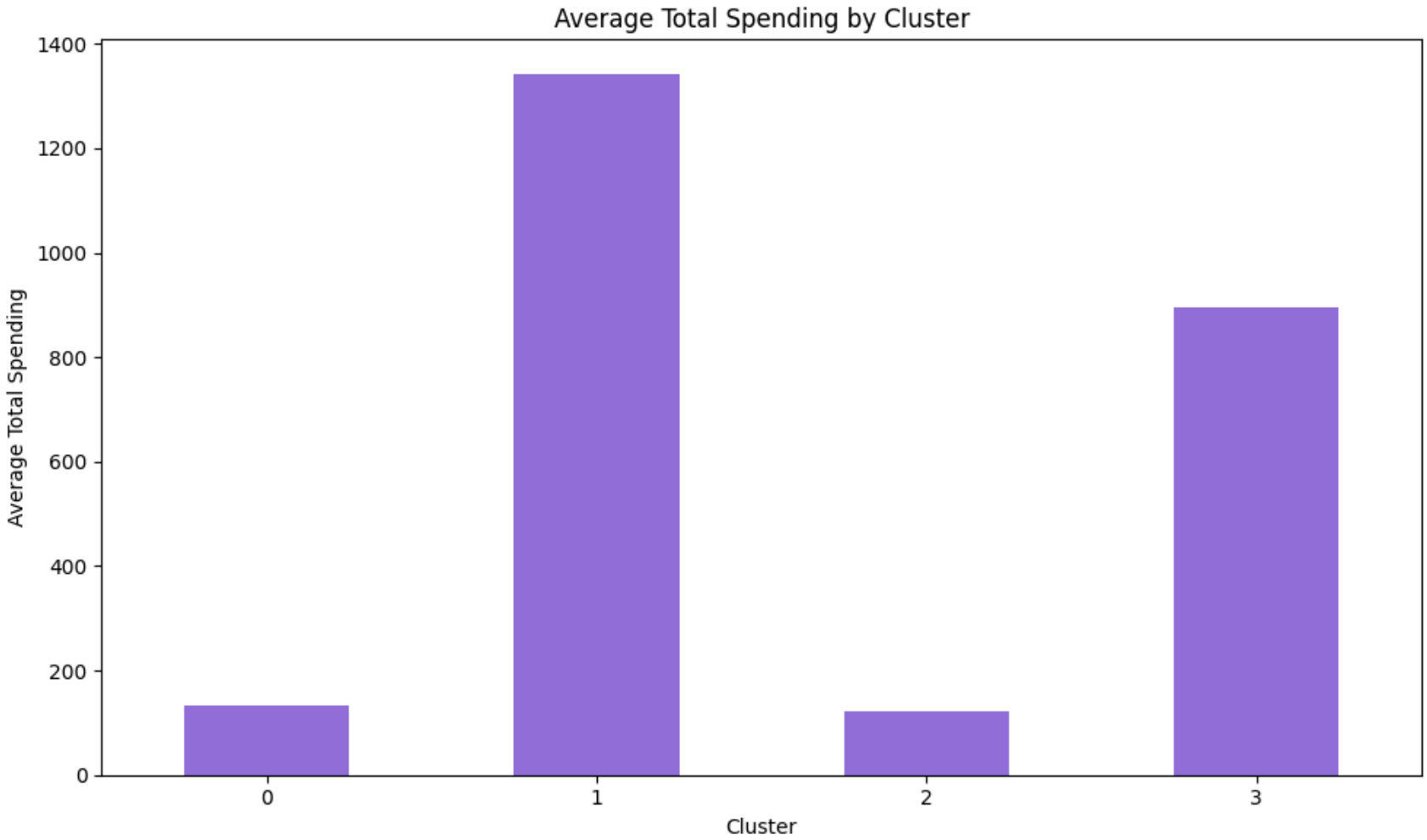
```
# Profile the clusters using average values
cluster_profile = df.groupby("Cluster")[[
    "Income",
    "Recency",
    "Total_Spending",
    "Total_Purchases",
    "NumWebVisitsMonth",
    "Response",
    "Total_Children",
    "Age"
]].mean().round(2)
```

```
print(cluster_profile)
```

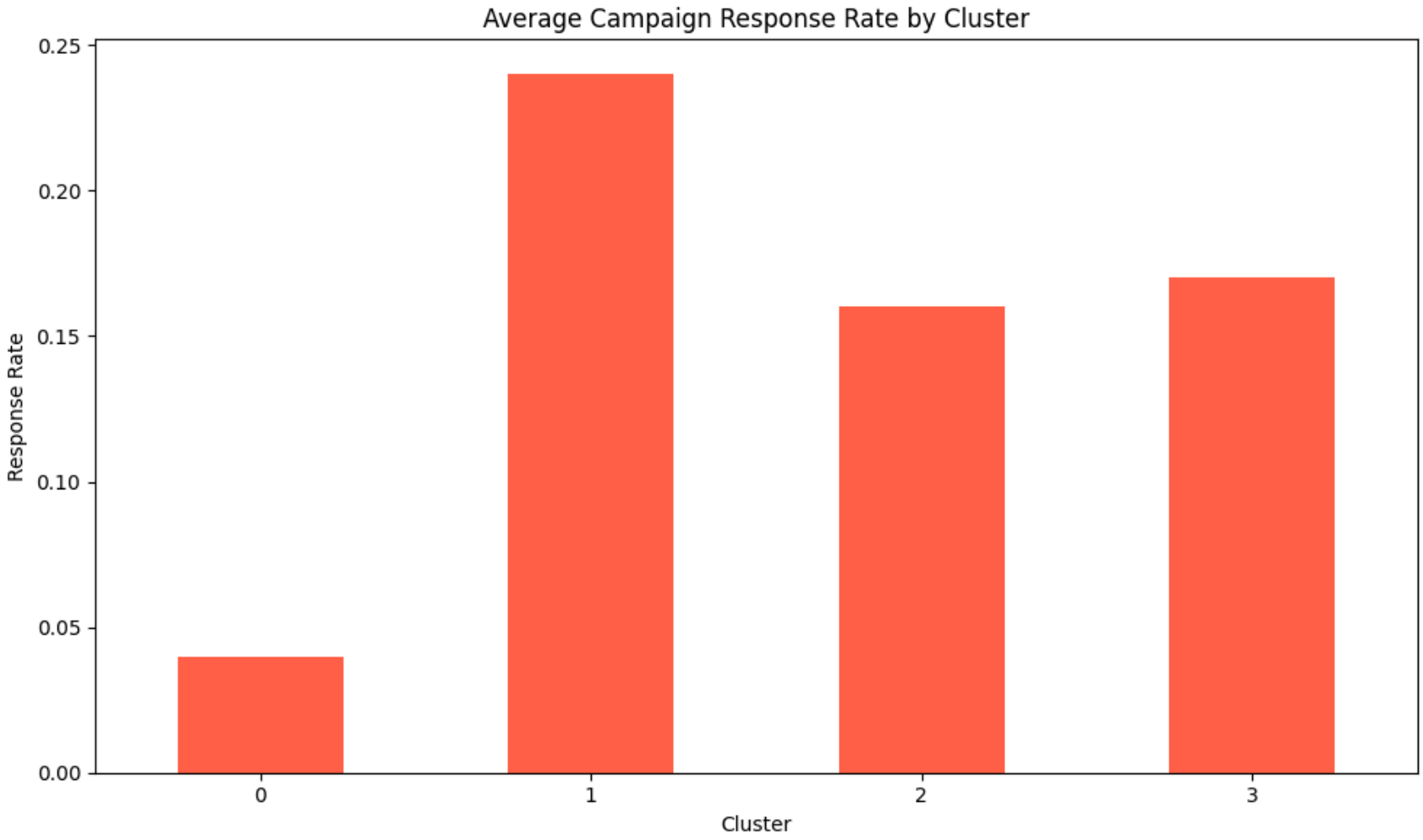
```
           Income  Recency  Total_Spending  Total_Purchases  \
Cluster
0        36938.64    74.98          133.22             6.74
1        78165.75    50.68         1341.26            18.96
2        34818.09    24.17          121.08             6.37
3        61218.34    47.48          895.84            19.02

         NumWebVisitsMonth  Response  Total_Children    Age
Cluster
0                     6.22      0.04            1.26  56.01
1                     2.39      0.24            0.27  58.56
2                     6.64      0.16            1.21  54.61
3                     5.89      0.17            1.01  59.43
```

In [23]:

```
# Plot: Average total spending by cluster
plt.figure(figsize=(10, 6))
cluster_profile["Total_Spending"].plot(kind="bar", color="mediumpurple")
plt.title("Average Total Spending by Cluster")
plt.xlabel("Cluster")
plt.ylabel("Average Total Spending")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

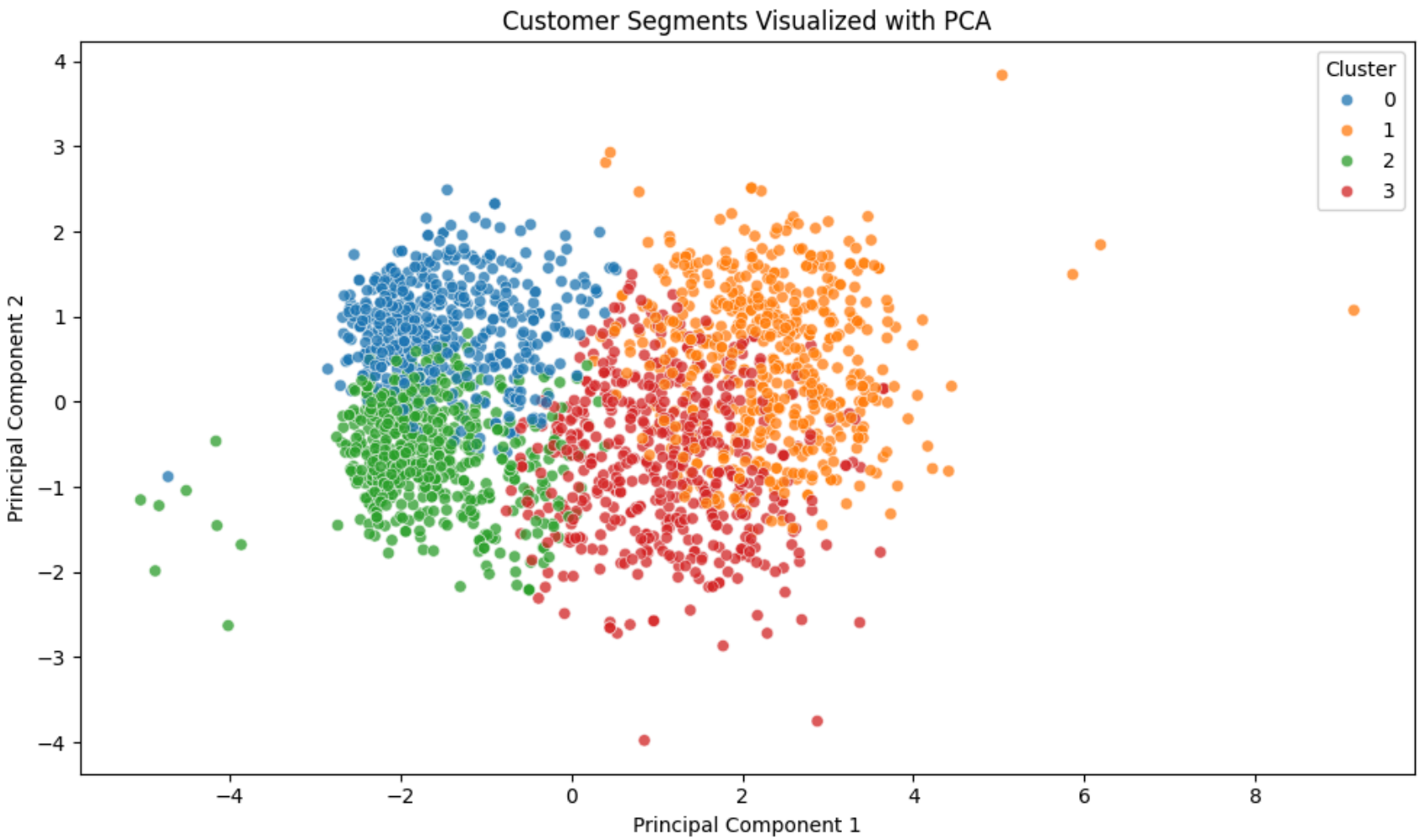


In [24]:
```
# Plot: Campaign response rate by cluster
plt.figure(figsize=(10, 6))
cluster_profile["Response"].plot(kind="bar", color="tomato")
plt.title("Average Campaign Response Rate by Cluster")
plt.xlabel("Cluster")
plt.ylabel("Response Rate")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



In [25]:
```
# Use PCA to reduce the clustering features to 2 dimensions for visualization
pca_model = PCA(n_components=2)
pca_components = pca_model.fit_transform(cluster_scaled)

pca_df = pd.DataFrame(pca_components, columns=["PC1", "PC2"])
pca_df["Cluster"] = df["Cluster"].values

# Plot: PCA projection of customer clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=pca_df,
```

```
    x="PC1",
    y="PC2",
    hue="Cluster",
    palette="tab10",
    alpha=0.75
)
plt.title("Customer Segments Visualized with PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Cluster")
plt.tight_layout()
plt.show()
```

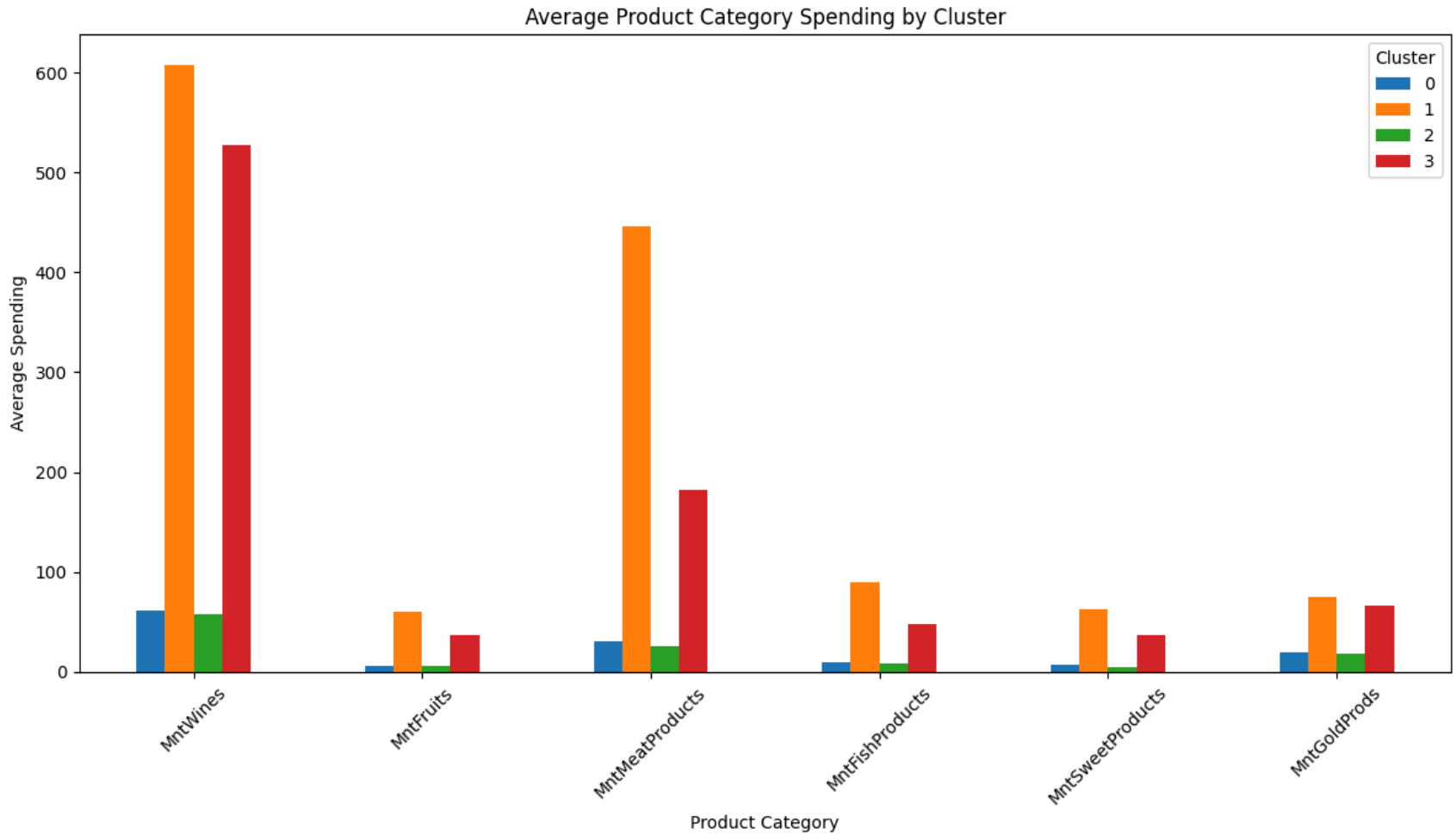


In [26]:
```
# Examine spending by product category across clusters
category_profile = df.groupby("Cluster")[spend_cols].mean().round(2)
print(category_profile)
```

```
         MntWines  MntFruits  MntMeatProducts  MntFishProducts  \
Cluster
0           60.91       6.30            30.44             9.14
1          607.65      60.06           446.35            89.91
2           57.96       5.48            26.10             8.08
3          527.25      36.13           182.52            47.15

         MntSweetProducts  MntGoldProds
Cluster
0                    6.75         19.68
1                   62.50         74.79
2                    5.09         18.36
3                   36.57         66.21
```

In [27]:

```
# Plot: Product spending profile by cluster
category_profile.T.plot(kind="bar", figsize=(12, 7))
plt.title("Average Product Category Spending by Cluster")
plt.xlabel("Product Category")
plt.ylabel("Average Spending")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [28]:
```
# Save cleaned dataset with cluster labels for appendix or export
df.to_csv("customer_segmentation_clustered_output.csv", index=False)

# Save cluster profile table
cluster_profile.to_csv("cluster_profile_summary.csv")

# Save category profile table
category_profile.to_csv("cluster_category_profile.csv")

print("Files saved successfully.")
print("customer_segmentation_clustered_output.csv")
```

```
print("cluster_profile_summary.csv")
print("cluster_category_profile.csv")
```

```
Files saved successfully.
customer_segmentation_clustered_output.csv
cluster_profile_summary.csv
cluster_category_profile.csv
```